

1 ● MongoDB Basics

MongoDB एउटा **NoSQL Database** हो। यसले डेटालाई Table को सट्टा **JSON जस्तै देखिने Documents** मा सेभ गर्छ। [\[1, 2, 3, 4\]](#)

- **Database:** धेरै वटा Collections को समूह।
 - **Collection:** SQL को 'Table' जस्तै, जहाँ धेरै Documents बस्छन् (जस्तै: `users`, `products`)।
 - **Document:** SQL को 'Row' जस्तै, जहाँ वास्तविक डाटा Key-Value pair मा बस्छ। [\[1, 2, 3\]](#)
-

2 ● Mongoose

Mongoose एउटा **ODM (Object Data Modeling)** लाइब्रेरी हो। यसले Node.js र MongoDB बीच पुलको काम गर्छ, जसले गर्दा डाटा म्यानेज गर्न सजिलो हुन्छ। [\[1, 2, 3, 4\]](#)

जडान गर्ने तरिका (Connection):

javascript

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/my_ecommerce')
  .then(() => console.log('Database connected successfully!'))
  .catch(err => console.error('Connection error:', err));

Use code with caution.
```

3 ● Schema & Models

- **Schema:** यसले डकुमेन्टको रूपरेखा वा 'Blueprint' कस्तो हुने भनी तय गर्छ (कुन फिल्ड रहने, डाटा टाइप के हुने)।
- **Model:** Schema मा आधारित एउटा Class हो, जसले डाटाबेससँग कुरा गर्छ (Data Create, Read गर्न)। [\[1, 2\]](#)

javascript

```
// १. Schema बनाउने
const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: Number,
  category: String,
  createdAt: { type: Date, default: Date.now }
});
```

```
});
```

```
// २. Model बनाउने
```

```
const Product = mongoose.model('Product', productSchema);
```

Use code with caution.

4 ● Validation (डाटा जाँच गर्ने)

डाटाबेस पठाउनु अघि डाटा सहि छ कि छैन भनी Schema ले नै चेक गर्छ।

javascript

```
const userSchema = new mongoose.Schema({
  username: { type: String, required: [true, 'नाम अनिवार्य छ!'] },
  email: { type: String, unique: true },
  age: { type: Number, min: 18, max: 65 } // १८ देखि ६५ वर्ष मात्र मिल्ने
});
```

Use code with caution.

5 ● CRUD Operations (मुख्य कामहरू)

डाटाबेसमा गरिने ४ वटा मुख्य कामहरू:

१. Create (डाटा सेभ गर्ने)

javascript

```
const newProduct = new Product({ name: 'Laptop', price: 1200, category:
'Electronics' });
await newProduct.save();
Use code with caution.
```

२. Read (डाटा खोज्ने)

javascript

```
const allProducts = await Product.find(); // सबै सामानहरू ल्याउने
const cheapProducts = await Product.find({ price: { $lt: 500 } }); // ५०० भन्दा सस्तो सामान
const oneProduct = await Product.findById('PRODUCT_ID'); // ID बाट खोज्ने
Use code with caution.
```

३. Update (डाटा सच्याउने)

javascript

```
await Product.findByIdAndUpdate('PRODUCT_ID', { price: 1100 });
Use code with caution.
```

४. Delete (डाटा हटाउने) [\[1, 2\]](#)

javascript

```
await Product.findByIdAndDelete('PRODUCT_ID');
Use code with caution.
```

6 Relationships & Populate

MongoDB मा एउटा डकुमेन्टलाई अर्को डकुमेन्टसँग जोड्न `ObjectId` र `ref` को प्रयोग गरिन्छ। [\[1, 2, 3, 4\]](#)

१. Relationship बनाउने (Order Schema मा User जोड्ने):

javascript

```
const orderSchema = new mongoose.Schema({
  productName: String,
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' } // User सँग जोडियो
});
const Order = mongoose.model('Order', orderSchema);
Use code with caution.
```

२. Populate (डाटा जोडेर ल्याउने):

साधारण `find()` गर्दा युजरको ID मात्र देखिन्छ। `populate()` गर्दा त्यो युजरको पूरा नाम र इमेल पनि सँगै आउँछ। [\[1, 2\]](#)

javascript

```
const orderDetails = await Order.find().populate('user');
// यसले अर्डरको विवरणसँगै अर्डर गर्ने मान्छेको प्रोफाइल पनि देखाउँछ।
Use code with caution.
```

7 ● Indexes (खोज्ने गति बढाउने)

यदि डाटाबेसमा लाखौं डाटाहरू छन् भने खोज्न समय लाग्छ। Indexing ले किताबको 'Index' पानाले जस्तै छिटो डाटा भेटाउन मद्दत गर्छ।

javascript

```
productSchema.index({ price: 1 }); // अब price बाट सामान खोज्दा एकदम फास्ट हुन्छ  
Use code with caution.
```

8 ● Aggregation Basics (डाटा फिल्टर र हिसाब गर्ने)

Aggregation भनेको धेरै डाटाहरूलाई विभिन्न चरण (Stages) बाट पास गराएर एउटै नतिजा निकाल्ने एडभान्स तरिका हो (जस्तै: Pipeline)। [\[1\]](#), [\[2\]](#)

उदाहरण (इलेक्ट्रोनक्स विधाका सामानहरूको औसत मूल्य निकाल्ने):

javascript

```
const stats = await Product.aggregate([  
  { $match: { category: 'Electronics' } }, // १. इलेक्ट्रोनक्स मात्र छान्ने  
  { $group: { _id: '$category', avgPrice: { $avg: '$price' } } } // २. समूह  
  बनाएर औसत निकाल्ने  
]);  
Use code with caution.
```